FECAP

Fundação Escola de Comércio Álvares Penteado
Ciência da Computação

GABRIEL CARVALHO MOTA
GUILHERME DE LIMA SIQUEIRA
RODRIGO LUIZ MENEZES DOS REIS
VITÓRIA LETÍCIA MACIEL DA SILVA

SISTEMAS EMBARCADOS ROBOTICA

Entrega 02 – Programação e Controle de Dispositivos no Arduino

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO 3
2 MATERIAIS E COMPONENTES UTILIZADOS 3
3 MONTAGEM DO CIRCUITO 3
3.1 Ligações do circuito 3
3.2 Passo a passo para reproduzir 4
4 FUNCIONAMENTO DO CÓDIGO 4
4.1 Modos de operação 5
4.2 Detecção e classificação do impacto 5
5 EVIDÊNCIA PRÁTICA 6
6 CONCLUSÃO 6
APÊNDICE A — CÓDIGO-FONTE COMPLETO 7

1 INTRODUÇÃO

Nesta segunda entrega, o circuito do Tobias, o "João Bobo" do projeto Smart Tennis Arena, deixou a simulação no Wokwi e foi montado fisicamente. O buzzer e o módulo microSD da simulação deram lugar ao DFPlayer Mini com alto-falante, e todo o circuito passou a ser alimentado pela bateria 18650 com step-up. O ESP32 lê o MPU6050, classifica a força de cada bolada e responde com som, luz e uma mensagem BLE ao aplicativo. A seguir são descritos os componentes, a montagem e o código, de modo que o circuito possa ser reproduzido.

2 MATERIAIS E COMPONENTES UTILIZADOS

O Quadro 1 relaciona os componentes utilizados na montagem física e a função de cada um no circuito.

Componente	Qtd	Função no circuito
ESP32 DevKit (ESP32 Dev Module)	1	Microcontrolador: lê o sensor, controla o LED e o áudio e se comunica por BLE
MPU6050 (GY-521)	1	Acelerômetro: mede a inclinação do boneco após o impacto
DFPlayer Mini (HW-247A) + cartão microSD (FAT32)	1	Reproduz os áudios de resposta gravados no cartão
Alto-falante 4 Ω / 3 W	1	Saída de som
LED verde + resistor 220 Ω	1	Indicador visual de que o impacto foi registrado
Resistor 1 kΩ	1	Em série na linha RX do DFPlayer; reduz ruído no áudio
Bateria 18650 (3,7 V, 3000 mAh) + step-up 5 V	1	Alimentação de todo o circuito
Protoboard e jumpers	—	Suporte e conexões elétricas entre os componentes

Quadro 1 — Componentes utilizados e suas funções. Fonte: elaborado pelos autores (2026).

3 MONTAGEM DO CIRCUITO

3.1 Ligações do circuito

A bateria alimenta o conversor step-up, cuja saída de 5 V vai ao pino VIN do ESP32 e à linha positiva da protoboard. Todos os GND são ligados entre si. O Quadro 2 apresenta as ligações de cada componente.

Componente	Pino do componente	Ligação
Step-up (saída 5 V)	OUT+ / OUT−	VIN do ESP32 / GND
MPU6050	VCC / GND	Linha positiva / linha negativa
MPU6050	SDA / SCL	GPIO 22 / GPIO 21
MPU6050	AD0	GND (endereço I²C 0x68)
DFPlayer Mini	VCC (pino 1) / GND (pino 7 ou 10)	5 V (VIN) / GND
DFPlayer Mini	RX (pino 2)	Resistor 1 kΩ → GPIO 17 (TX2)
DFPlayer Mini	SPK_1 (pino 6) / SPK_2 (pino 8)	Alto-falante 4 Ω
LED verde	Ânodo / cátodo	Resistor 220 Ω → GPIO 2 / GND

Quadro 2 — Ligações do circuito físico. Fonte: elaborado pelos autores (2026).

A Figura 1 mostra o circuito montado, com todos os componentes ligados.

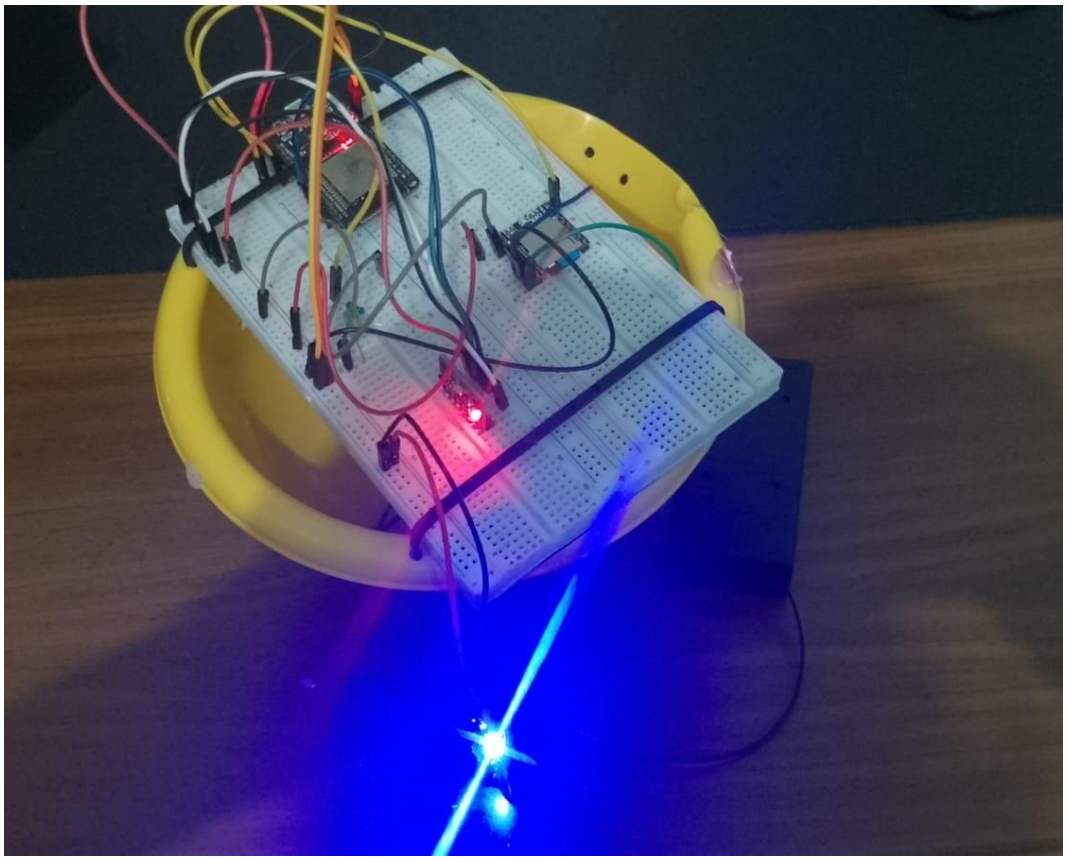


Figura 1 — Circuito montado. Fonte: elaborado pelos autores (2026).

3.2 Passo a passo para reproduzir

1. Faça as ligações do Quadro 2 com a bateria desconectada.
2. Grave no cartão microSD, dentro da pasta */mp3/*, os arquivos 0001.mp3 (fraco), 0002.mp3 (médio) e 0004.mp3 (K.O.).
3. Grave o firmware pelo PlatformIO (placa esp32dev, framework Arduino, monitor serial a 115200 baud). Não há bibliotecas externas: Wire e BLE já fazem parte do núcleo do ESP32.
4. Ligue a bateria com o boneco parado e em pé, pois o firmware mede a posição de repouso no primeiro segundo. Três piscadas do LED indicam que o sistema está pronto.

4 FUNCIONAMENTO DO CÓDIGO

O firmware foi escrito em C++ (framework Arduino) e gravado pelo PlatformIO. O código-fonte completo e comentado está no Apêndice A; esta seção explica sua lógica.

4.1 Modos de operação

O programa tem três modos, comandados pelo aplicativo via BLE. O ESP32 anuncia o nome TOBIAS e recebe comandos de texto pelo serviço Nordic UART.

1. **Aguardando** — modo inicial: o Tobias fica em silêncio e ignora batidas até receber um comando.
2. **Calibração** (CALIBRAR) — espera cinco batidas, sem limite de tempo, e informa o progresso ao aplicativo.
3. **Treino** (TREINO:quantidade:intervalo:dificuldade) — conta os acertos e registra erro se não houver impacto no intervalo (mínimo de 3 s). A dificuldade, em graus, soma-se aos limites da Tabela 1. STOP encerra a sessão.

Ao fim de cada sessão, o ESP32 envia um relatório com acertos, erros, precisão e ângulos médio e máximo, e volta ao modo Aguardando.

4.2 Detecção e classificação do impacto

A cada 10 ms (cerca de 100 leituras por segundo), o ESP32 lê o acelerômetro pelo barramento I²C e calcula o ângulo de inclinação do boneco. Ele é obtido pelo produto escalar entre o vetor de aceleração atual e o de repouso medido na inicialização, $\theta = \arccos(a \cdot ref / |a|)$, o que torna a medida independente da posição do sensor. O impacto é tratado por uma máquina de quatro estados:

1. **Repouso pronto** — aguarda o ângulo atingir 22°.
2. **Capturando pico** — durante 350 ms, registra o maior ângulo alcançado e, em seguida, classifica a batida conforme a Tabela 1.
3. **Tocando áudio** — toca o som da classe, acende o LED e envia o resultado ao aplicativo em JSON. O som dura 1,8 s, com redução gradual de volume nos últimos 0,6 s.
4. **Esperando estabilizar** — só volta ao primeiro estado quando o ângulo cai abaixo de 14°, para que o balanço de volta do boneco não seja contado como nova batida.

Classe	Ângulo de pico	Áudio tocado
Fraco	22° a 38°	0001.mp3
Médio	38° a 58°	0002.mp3
Forte (K.O.)	Acima de 58°	0004.mp3

Tabela 1 — Classificação do impacto pelo ângulo de pico. Fonte: elaborado pelos autores (2026).

5 EVIDÊNCIA PRÁTICA

A Figura 2 mostra as mensagens que o Tobias enviou por BLE a um aplicativo de terminal serial Bluetooth no celular, durante um treino de 12 lances com intervalo de 5 s (comando TREINO:12:5000). O primeiro lance foi registrado como erro por tempo esgotado,

e os 11 seguintes como acertos: 4 fracos, 4 médios e 3 fortes (K.O.). O relatório final indica precisão de 91%, ângulo médio de 42,8° e máximo de 66,8°.

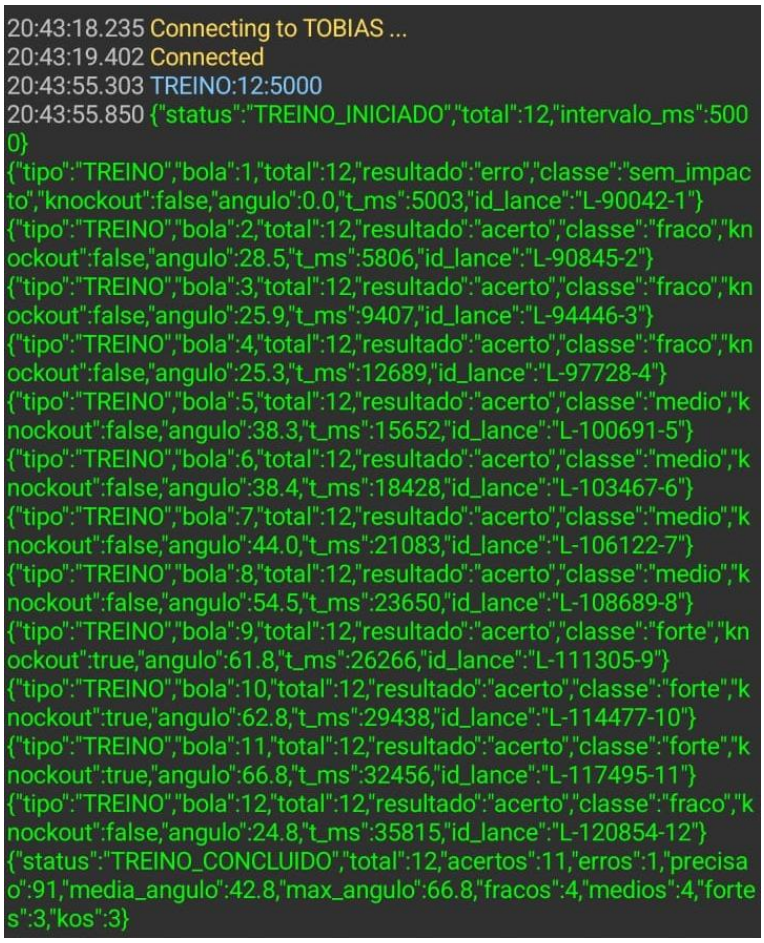


Figura 2 — Mensagens BLE recebidas no celular durante um treino de 12 lances. Fonte: elaborado pelos autores (2026).

Vídeo do circuito em funcionamento: https://drive.google.com/drive/folders/1AaJh-KtsCsyEaz6pirNKTcfTmJsdOJMG?usp=sharing

6 CONCLUSÃO

A montagem física confirmou o funcionamento integrado do sensor, do áudio, do LED e da comunicação BLE, alimentados pela bateria 18650. No treino da Figura 2, cada um dos 11 impactos gerou um único evento, distribuído nas três classes de intensidade, e o lance sem impacto foi registrado como erro. Isso indica que a medição por ângulo e a espera pela estabilização do boneco evitam a contagem duplicada de batidas, apontada como risco na Entrega 1.

APÊNDICE A — CÓDIGO-FONTE COMPLETO

A seguir estão os dois arquivos do projeto no PlatformIO: a configuração (platformio.ini) e o firmware (src/main.cpp), reproduzidos na íntegra.

A.1 platformio.ini

```
; PlatformIO Project Configuration File - Entrega 2
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
```

A.2 src/main.cpp

```
#include <Wire.h>
#include <math.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

// --- MAPEAMENTO DE PINOS ---
#define PIN_SDA 22        // MPU-6050 SDA (Fio Azul)
#define PIN_SCL 21        // MPU-6050 SCL (Fio Verde)
#define PIN_LED_VERDE 2   // LED Verde (GPIO 2)

#define RXD2 16
#define TXD2 17           // ESP32 TX2 -> Resistor 1kΩ -> RX do DFPlayer

#define MPU_ADDR 0x68

// --- UUIDs BLE PADRÃO NORDIC UART SERVICE ---
#define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

BLEServer *pServer = NULL;
BLECharacteristic *pTxCharacteristic = NULL;
bool deviceConnected = false;

// --- LIMIARES FÍSICOS DE INCLINAÇÃO (EM GRAUS) ---
const float ANGULO_GATILHO = 22.0; // Mínimo de 22° para validar pancada
const float ANGULO_MEDIO   = 38.0; // 22° a 38° = FRACO | 38° a 58° = MÉDIO
const float ANGULO_KO      = 58.0; // Acima de 58° = KO (FORTE)
const float ANGULO_REPOUSO = 14.0; // Abaixo de 14° = Tobias voltou a ficar em pé
float calibDificuldade     = 0.0;  // Dificuldade dinâmica configurada pela calibração (graus
adicionais)

// Estados da Máquina Física do Tobias
enum EstadoFisico { REPOUSO_PRONTO, CAPTURANDO_PICO, TOCANDO_AUDIO, ESPERANDO_ESTABILIZAR };
EstadoFisico estadoFisico = REPOUSO_PRONTO;

// Modos de Operação do Sistema
enum ModoOperacao { MODO_AGUARDANDO, MODO_CALIBRACAO, MODO_TREINO };
ModoOperacao modoAtual = MODO_AGUARDANDO; // Inicia em STANDBY silencioso

// Vetor de Referência da Gravidade (calibração automática da verticalidade no setup)
float refX = 0, refY = 0, refZ = 1.0;

unsigned long tempoInicioImpacto = 0;
```

```
unsigned long tempoInicioAudio = 0;
unsigned long ultimoAjusteVolume = 0;
unsigned long tempoInicioSessao = 0;

// Temporização de Lances e Timeout (Erros)
unsigned long tempoInicioJanelaLance = 0;
unsigned long intervaloLanceMs = 3000;
const unsigned long INTERVALO_MINIMO_MS = 3000;

float picoAnguloRegistrado = 0.0;
const uint8_t VOLUME_PADRAO = 15; // Volume confortável (0 a 30)

// Contadores e Estatísticas
int bolaAtual = 0;
int totalBolasSessao = 0;
int contAcertos = 0;
int contErros = 0;
int contFraco = 0;
int contMedio = 0;
int contForte = 0;
int contKO = 0;
float somaAngulos = 0;
float maiorAngulo = 0;

// --- PROTÓTIPOS DE FUNÇÕES ---
void enviarBLE(String msg);
void enviarComandoDFPlayer(uint8_t comando, uint8_t p1, uint8_t p2);
void tocarFaixaMP3(int faixa);
void setVolume(uint8_t vol);
void pararAudio();
void lerAcelerometroRaw(float *x, float *y, float *z, bool acumular = false);
float calcularAnguloInclinacao(float ax, float ay, float az);
void iniciarCalibracao(int total);
void iniciarTreino(int total, unsigned long intervaloMs, float calib = 0.0);
void registrarErroTimeout();
void finalizarSessao();

// --- IMPLEMENTAÇÃO DE ÁUDIO DFPLAYER MINI ---

void enviarComandoDFPlayer(uint8_t comando, uint8_t p1, uint8_t p2) {
  uint8_t buffer[10] = {0x7E, 0xFF, 0x06, comando, 0x00, p1, p2, 0x00, 0x00, 0xEF};
  uint16_t checksum = 0;
  for (int i = 1; i < 7; i++) checksum += buffer[i];
  checksum = -checksum;
  buffer[7] = (uint8_t)(checksum >> 8);
  buffer[8] = (uint8_t)checksum;

  Serial2.write(buffer, 10);
  delay(30);
}

void tocarFaixaMP3(int faixa) {
  Serial.printf(" [AUDIO] Disparando faixa 000%d.mp3 (Volume %d)\n", faixa, VOLUME_PADRAO);
  enviarComandoDFPlayer(0x12, 0x00, (uint8_t)faixa);
}

void setVolume(uint8_t vol) {
  if (vol > 30) vol = 30;
  enviarComandoDFPlayer(0x06, 0x00, vol);
}

void pararAudio() {
  enviarComandoDFPlayer(0x16, 0x00, 0x00);
```

```
}

// --- COMUNICAÇÃO BLUETOOTH BLE ---

void enviarBLE(String msg) {
  if (deviceConnected && pTxCharacteristic != NULL) {
    // '\n' marca o fim de cada JSON para o app (que junta os pacotes BLE por linha)
    String linha = msg + "\n";
    pTxCharacteristic->setValue(linha.c_str());
    pTxCharacteristic->notify();
  }
  Serial.print("📱 [BLE TX] -> ");
  Serial.println(msg);
}

class MyServerCallbacks: public BLEServerCallbacks {
  void onConnect(BLEServer* pServer) {
    deviceConnected = true;
    Serial.println("📱 Celular conectado ao Tobias!");
    enviarBLE("{\"status\":\"CONECTADO\",\"dispositivo\":\"TOBIAS\",\"modo\":\"AGUARDANDO\"}");
  }
  void onDisconnect(BLEServer* pServer) {
    deviceConnected = false;
    Serial.println("📱 Celular desconectado.");
    pServer->startAdvertising();
  }
};

class MyRxCallbacks: public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *pCharacteristic) {
    String comando = pCharacteristic->getValue().c_str();
    comando.trim();
    Serial.print("📩 [BLE RX] Comando recebido: ");
    Serial.println(comando);

    // 1. Iniciar Calibração (App envia "CALIBRAR" ou "CALIBRAR:5")
    if (comando.startsWith("CALIBRAR")) {
      int bolas = 5;
      int index = comando.indexOf(':');
      if (index != -1) {
        bolas = comando.substring(index + 1).toInt();
        if (bolas <= 0) bolas = 5;
      }
      iniciarCalibracao(bolas);
    }
    // 2. Iniciar Treino (App envia "TREINO:15:5" ou "TREINO:12:3:8.5"...)
    else if (comando.startsWith("TREINO")) {
      int qtd = 15;
      unsigned long intervalo = INTERVALO_MINIMO_MS;
      float calib = 0.0;

      int primeiroDoisPontos = comando.indexOf(':');
      if (primeiroDoisPontos != -1) {
        int segundoDoisPontos = comando.indexOf(':', primeiroDoisPontos + 1);
        if (segundoDoisPontos != -1) {
          qtd = comando.substring(primeiroDoisPontos + 1, segundoDoisPontos).toInt();
          int terceiroDoisPontos = comando.indexOf(':', segundoDoisPontos + 1);
          if (terceiroDoisPontos != -1) {
            unsigned long val = comando.substring(segundoDoisPontos + 1,
terceiroDoisPontos).toInt();
            intervalo = (val < 100) ? (val * 1000) : val;
            calib = comando.substring(terceiroDoisPontos + 1).toFloat();
          } else {
```

```
            unsigned long val = comando.substring(segundoDoisPontos + 1).toInt();
            intervalo = (val < 100) ? (val * 1000) : val;
          }
        } else {
          qtd = comando.substring(primeiroDoisPontos + 1).toInt();
        }
      }

      if (qtd <= 0) qtd = 15;
      if (intervalo < INTERVALO_MINIMO_MS) intervalo = INTERVALO_MINIMO_MS;

      iniciarTreino(qtd, intervalo, calib);
    }
    // 3. Cancelar Sessão / Voltar ao Standby Silencioso
    else if (comando == "STOP" || comando == "CANCELAR" || comando == "RESET") {
      modoAtual = MODO_AGUARDANDO;
      bolaAtual = 0;
      totalBolasSessao = 0;
      pararAudio();
      setVolume(VOLUME_PADRAO);
      digitalWrite(PIN_LED_VERDE, LOW);
      enviarBLE("{\"status\":\"CANCELADO\",\"modo\":\"AGUARDANDO\"}");
      Serial.println("🔴 Sessão cancelada. Tobias em STANDBY silencioso.");
    }
    // 4. Ping / Status
    else if (comando == "PING") {
      String mStr = (modoAtual == MODO_CALIBRACAO) ? "CALIBRACAO" : (modoAtual == MODO_TREINO) ?
"TREINO" : "AGUARDANDO";
      enviarBLE("{\"status\":\"PONG\",\"modo\":\"" + mStr + "\",\"bola_atual\":" +
String(bolaAtual) + "}");
    }
  }
};

void iniciarCalibracao(int total) {
  modoAtual = MODO_CALIBRACAO;
  totalBolasSessao = total;
  bolaAtual = 0;
  contAcertos = contErros = 0;
  contFraco = contMedio = contForte = contKO = 0;
  somaAngulos = 0;
  maiorAngulo = 0;
  calibDificuldade = 0.0; // Na calibração sempre usa os limiares base
  tempoInicioSessao = millis();

  digitalWrite(PIN_LED_VERDE, HIGH); delay(200); digitalWrite(PIN_LED_VERDE, LOW);
  tocarFaixaMP3(1); // Som curto de sinalização
  enviarBLE("{\"status\":\"CALIB_INICIADA\",\"total\":" + String(totalBolasSessao) + "}");
  Serial.printf("🔑 Modo CALIBRAÇÃO iniciado: %d batidas aguardadas.\n", totalBolasSessao);
}

void iniciarTreino(int total, unsigned long intervaloMs, float calib) {
  modoAtual = MODO_TREINO;
  totalBolasSessao = total;
  intervaloLanceMs = intervaloMs;
  calibDificuldade = (calib > 0.0) ? calib : 0.0;
  bolaAtual = 0;
  contAcertos = contErros = 0;
  contFraco = contMedio = contForte = contKO = 0;
  somaAngulos = 0;
  maiorAngulo = 0;
  tempoInicioSessao = millis();
  tempoInicioJanelaLance = millis();
```

```
  Serial.printf("🎯 Treino Iniciado: %d lances | Intervalo: %lu ms | Dificuldade: +%.1f°\n",
                totalBolasSessao, intervaloLanceMs, calibDificuldade);

  for (int i = 0; i < 2; i++) {
    digitalWrite(PIN_LED_VERDE, HIGH); delay(100);
    digitalWrite(PIN_LED_VERDE, LOW);  delay(100);
  }
  tocarFaixaMP3(2);

  String json = "{\"status\":\"TREINO_INICIADO\",\"total\":" + String(totalBolasSessao) +
                ",\"intervalo_ms\":" + String(intervaloLanceMs) +
                ",\"dificuldade\":" + String(calibDificuldade, 1) + "}";
  enviarBLE(json);
}

void registrarErroTimeout() {
  bolaAtual++;
  contErros++;
  unsigned long tMs = (tempoInicioSessao > 0) ? (millis() - tempoInicioSessao) : millis();
  String idLance = "L-" + String(millis()) + "-" + String(bolaAtual);

  Serial.printf("❌ [ERRO] Lance %d/%d: Tempo limite expirado (%lu ms) sem impacto!\n",
                bolaAtual, totalBolasSessao, intervaloLanceMs);

  digitalWrite(PIN_LED_VERDE, HIGH); delay(60); digitalWrite(PIN_LED_VERDE, LOW);

  String jsonErro = String("{\"tipo\":\"TREINO\",\"bola\":") + String(bolaAtual) +
                    ",\"total\":" + String(totalBolasSessao) +
                    ",\"resultado\":\"erro\"" +
                    ",\"classe\":\"sem_impacto\"" +
                    ",\"knockout\":false" +
                    ",\"angulo\":0.0" +
                    ",\"t_ms\":" + String(tMs) +
                    ",\"id_lance\":\"" + idLance + "\"}";
  enviarBLE(jsonErro);

  tempoInicioJanelaLance = millis();

  if (bolaAtual >= totalBolasSessao) {
    finalizarSessao();
  }
}

void finalizarSessao() {
  float media = (contAcertos > 0) ? (somaAngulos / contAcertos) : 0;
  int precisao = (totalBolasSessao > 0) ? ((contAcertos * 100) / totalBolasSessao) : 0;
  String tipoSessao = (modoAtual == MODO_CALIBRACAO) ? "CALIB_CONCLUIDA" : "TREINO_CONCLUIDO";

  String jsonRelatorio = String("{\"status\":\"") + tipoSessao + "\"" +
                         ",\"total\":" + String(totalBolasSessao) +
                         ",\"acertos\":" + String(contAcertos) +
                         ",\"erros\":" + String(contErros) +
                         ",\"precisao\":" + String(precisao) +
                         ",\"media_angulo\":" + String(media, 1) +
                         ",\"max_angulo\":" + String(maiorAngulo, 1) +
                         ",\"fracos\":" + String(contFraco) +
                         ",\"medios\":" + String(contMedio) +
                         ",\"fortes\":" + String(contForte) +
                         ",\"kos\":" + String(contKO) +
                         ",\"dificuldade\":" + String(calibDificuldade, 1) + "}";
  enviarBLE(jsonRelatorio);
```

```
  // Retorna imediatamente para o MODO_AGUARDANDO silencioso
  modoAtual = MODO_AGUARDANDO;
  bolaAtual = 0;
  totalBolasSessao = 0;
  digitalWrite(PIN_LED_VERDE, LOW);
  Serial.println("🔒 Sessão finalizada. Tobias retornou ao STANDBY silencioso.");
}

// --- MPU-6050 ---

void lerAcelerometroRaw(float *x, float *y, float *z, bool acumular) {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom((uint16_t)MPU_ADDR, (uint8_t)6, true);

  if (Wire.available() >= 6) {
    int16_t rawX = (Wire.read() << 8) | Wire.read();
    int16_t rawY = (Wire.read() << 8) | Wire.read();
    int16_t rawZ = (Wire.read() << 8) | Wire.read();

    float ax = (float)rawX / 2048.0;
    float ay = (float)rawY / 2048.0;
    float az = (float)rawZ / 2048.0;

    if (acumular) {
      *x += ax; *y += ay; *z += az;
    } else {
      *x = ax; *y = ay; *z = az;
    }
  }
}

float calcularAnguloInclinacao(float ax, float ay, float az) {
  float mod = sqrt(ax * ax + ay * ay + az * az);
  if (mod < 0.1) return 0.0;
  float dot = (ax * refX + ay * refY + az * refZ) / mod;
  if (dot > 1.0) dot = 1.0;
  if (dot < -1.0) dot = -1.0;
  return acos(dot) * (180.0 / 3.14159265);
}

// --- SETUP ---

void setup() {
  Serial.begin(115200);
  delay(500);

  pinMode(PIN_LED_VERDE, OUTPUT);
  digitalWrite(PIN_LED_VERDE, LOW);

  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  Wire.begin(PIN_SDA, PIN_SCL);
  Wire.setTimeOut(50);

  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x6B);
  Wire.write(0x00);
  Wire.endTransmission();

  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x1C);
```

```
  Wire.write(0x18); // 16G
  Wire.endTransmission();

  Serial.println("\n--- CALIBRANDO VERTICALIDADE DE REPOUSO DO TOBIAS ---");
  float somaX = 0, somaY = 0, somaZ = 0;
  for (int i = 0; i < 50; i++) {
    lerAcelerometroRaw(&somaX, &somaY, &somaZ, true);
    delay(20);
  }
  float modRef = sqrt(somaX * somaX + somaY * somaY + somaZ * somaZ);
  refX = somaX / modRef;
  refY = somaY / modRef;
  refZ = somaZ / modRef;

  BLEDevice::init("TOBIAS");
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService *pService = pServer->createService(SERVICE_UUID);
  pTxCharacteristic = pService->createCharacteristic(CHARACTERISTIC_UUID_TX,
BLECharacteristic::PROPERTY_NOTIFY);
  pTxCharacteristic->addDescriptor(new BLE2902());

  BLECharacteristic *pRxCharacteristic = pService->createCharacteristic(CHARACTERISTIC_UUID_RX,
BLECharacteristic::PROPERTY_WRITE);
  pRxCharacteristic->setCallbacks(new MyRxCallbacks());

  pService->start();

  // Anuncia o UUID do serviço NUS junto com o nome, para o app encontrar o Tobias no scan
  BLEAdvertising *pAdvertising = pServer->getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);
  pAdvertising->start();

  Serial.println("Inicializando DFPlayer Mini...");
  delay(1500);
  setVolume(VOLUME_PADRAO); // Volume 15

  // 3 piscadas confirmando que o sistema ligou
  for (int i = 0; i < 3; i++) {
    digitalWrite(PIN_LED_VERDE, HIGH); delay(100);
    digitalWrite(PIN_LED_VERDE, LOW);  delay(100);
  }

  Serial.println("============================================================");
  Serial.println("  SISTEMA TOBIAS ONLINE - EM STANDBY SILENCIOSO           ");
  Serial.println("  Aguardando comando do App ('TREINO' ou 'CALIBRAR')...   ");
  Serial.println("============================================================");
}

// --- LOOP PRINCIPAL ---

void loop() {
  // REGRA DE OURO: SE NÃO ESTIVER EM TREINO NEM CALIBRAÇÃO, FICA 100% SILENCIOSO!
  // Nenhuma batida, balanço ou toque emite som ou acende LED no modo de espera.
  if (modoAtual == MODO_AGUARDANDO) {
    delay(40);
    return;
  }

  float ax = 0, ay = 0, az = 0;
  lerAcelerometroRaw(&ax, &ay, &az);
```

```
float anguloAtual = calcularAnguloInclinacao(ax, ay, az);

// 1. CHECAGEM DE TIMEOUT (APENAS NO MODO TREINO)
// No modo calibração NÃO há timeout: o atleta bate as 5 bolas no tempo dele
if (modoAtual == MODO_TREINO && estadoFisico == REPOUSO_PRONTO) {
  if (millis() - tempoInicioJanelaLance >= intervaloLanceMs) {
    registrarErroTimeout();
  }
}

// 2. MÁQUINA DE ESTADOS FÍSICA
switch (estadoFisico) {

  case REPOUSO_PRONTO:
    if (anguloAtual >= ANGULO_GATILHO) {
      tempoInicioImpacto = millis();
      picoAnguloRegistrado = anguloAtual;
      estadoFisico = CAPTURANDO_PICO;
    }
    break;

  case CAPTURANDO_PICO:
    if (anguloAtual > picoAnguloRegistrado) {
      picoAnguloRegistrado = anguloAtual;
    }

    if (millis() - tempoInicioImpacto >= 350) {
      String classe = "fraco";
      bool knockout = false;
      int somTocar = 1;

      // Limiares ajustados dinamicamente com base na calibração do jogador
      float limiarKO = ANGULO_KO + calibDificuldade;
      float limiarMedio = ANGULO_MEDIO + calibDificuldade;

      if (picoAnguloRegistrado >= limiarKO) {
        classe = "forte";
        knockout = true;
        somTocar = 4; // 0004.mp3 (KO)
        contForte++;
        contKO++;
      }
      else if (picoAnguloRegistrado >= limiarMedio) {
        classe = "medio";
        knockout = false;
        somTocar = 2; // 0002.mp3 (Médio)
        contMedio++;
      }
      else {
        classe = "fraco";
        knockout = false;
        somTocar = 1; // 0001.mp3 (Fraco)
        contFraco++;
      }

      somaAngulos += picoAnguloRegistrado;
      if (picoAnguloRegistrado > maiorAngulo) maiorAngulo = picoAnguloRegistrado;

      // Dispara o som e acende o LED
      setVolume(VOLUME_PADRAO);
      tocarFaixaMP3(somTocar);
      digitalWrite(PIN_LED_VERDE, HIGH);
```

```
        unsigned long tMs = (tempoInicioSessao > 0) ? (millis() - tempoInicioSessao) : millis();
        bolaAtual++;
        contAcertos++;

        String tipoMsg = (modoAtual == MODO_CALIBRACAO) ? "CALIB" : "TREINO";
        String idLance = "L-" + String(millis()) + "-" + String(bolaAtual);

        String jsonLance = String("{\"tipo\":\"") + tipoMsg + "\"" +
                           ",\"bola\":" + String(bolaAtual) +
                           ",\"total\":" + String(totalBolasSessao) +
                           ",\"resultado\":\"acerto\"" +
                           ",\"classe\":\"" + classe + "\"" +
                           ",\"knockout\":" + (knockout ? "true" : "false") +
                           ",\"angulo\":" + String(picoAnguloRegistrado, 1) +
                           ",\"t_ms\":" + String(tMs) +
                           ",\"id_lance\":\"" + idLance + "\"}";
        enviarBLE(jsonLance);

        tempoInicioJanelaLance = millis(); // Reinicia janela do próximo lance
        tempoInicioAudio = millis();
        ultimoAjusteVolume = millis();
        estadoFisico = TOCANDO_AUDIO;
      }
      break;

    case TOCANDO_AUDIO: {
      unsigned long tempoDecorrido = millis() - tempoInicioAudio;

      if (tempoDecorrido >= 1200 && tempoDecorrido < 1800) {
        if (millis() - ultimoAjusteVolume >= 100) {
          ultimoAjusteVolume = millis();
          float progresso = (float)(tempoDecorrido - 1200) / 600.0;
          uint8_t volFade = VOLUME_PADRAO - (uint8_t)(progresso * (VOLUME_PADRAO - 2));
          setVolume(volFade);
        }
      }

      if (tempoDecorrido >= 1800) {
        pararAudio();
        setVolume(VOLUME_PADRAO);
        digitalWrite(PIN_LED_VERDE, LOW);

        if (bolaAtual >= totalBolasSessao) {
          finalizarSessao();
        }

        estadoFisico = ESPERANDO_ESTABILIZAR;
      }
      break;
    }

    case ESPERANDO_ESTABILIZAR:
      if (anguloAtual < ANGULO_REPOUSO) {
        estadoFisico = REPOUSO_PRONTO;
      }
      break;
  }

  delay(10);
}
```